

CHAPTER 03

OpenCV 주요 클래스

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

3장 OpenCV 주요 클래스

3.1 기본 자료형 클래스

3.2 Mat 클래스

3.3 Vec과 Scalar 클래스

3.4 InputArray와 OutputArray 클래스

3.2 Mat 클래스

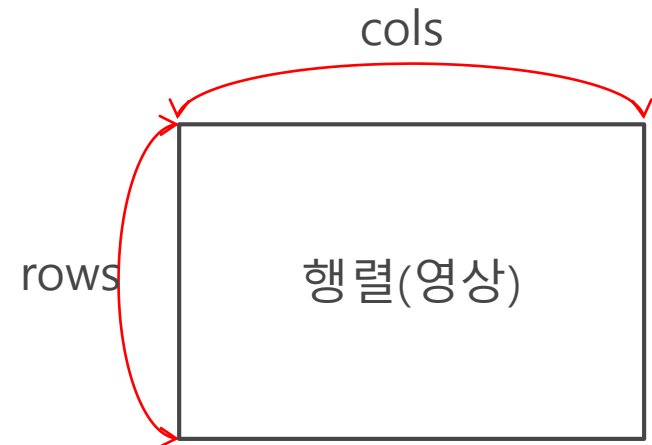
행렬 정보 참조하기

- Mat 객체에서 가장 자주 참조하는 정보는 행렬(영상)의 크기 정보임
- Mat::rows 멤버 변수는 행렬의 행 개수를 나타냄
- Mat::cols 멤버 변수는 행렬의 열 개수를 나타냄
- Mat 객체에 영상이 저장되어 있는 경우라면 Mat::rows는 영상의 세로 픽셀 크기이고 Mat::cols는 영상의 가로 픽셀 크기를 나타냄
- 이들 멤버 변수는 모두 public 접근 지시자로 선언되어 있기 때문에 클래스 외부에서도 자유롭게 접근할 수 있음

```
Mat img1 = imread("lenna.bmp");
```

```
cout << "Width: " << img1.cols << endl;
cout << "Height: " << img1.rows << endl;
```

```
Width: 512
Height: 512
```



행렬 정보 참조하기

- 크기 정보 외에 다른 행렬 정보는 Mat 클래스에서 제공하는 멤버 함수를 이용하여 참조할 수 있음

표 3-2 Mat 클래스 정보 참조를 위한 멤버 함수

Mat 클래스 멤버 함수	설명
<code>int Mat::channels() const;</code>	행렬의 채널 수를 반환합니다.
<code>int Mat::depth() const;</code>	행렬의 깊이를 반환합니다. (예) CV_8U, CV_32F
<code>size_t Mat::elemSize() const;</code>	한 개의 원소가 차지하는 메모리 크기를 바이트 단위로 반환합니다 (CV_32SC3 타입 행렬의 경우 $4 \times 3 = 12$ 를 반환).
<code>size_t Mat::elemSize1() const;</code>	하나의 채널에서 한 개의 원소가 차지하는 메모리 크기를 바이트 단위로 반환합니다(CV_32SC3 타입 행렬의 경우 4를 반환).

행렬 정보 참조하기

표 3-2 Mat 클래스 정보 참조를 위한 멤버 함수

Mat 클래스 멤버 함수	설명
<code>bool Mat::empty() const;</code>	비어 있는 행렬이면 true를 반환합니다.
<code>bool Mat::isContinuous() const;</code>	각 행의 원소가 연속적으로 저장되어 있으면 true를 반환합니다.
<code>bool Mat::isSubmatrix() const;</code>	행렬이 다른 행렬의 부분 행렬이면 true를 반환합니다.
<code>Size Mat::size() const;</code>	행렬 크기를 Size 타입으로 반환합니다. ¹¹
<code>size_t Mat::total() const;</code>	전체 원소 개수를 반환합니다.
<code>int Mat::type() const;</code>	행렬의 타입을 반환합니다. (예) CV_32FC1, CV_8UC3

행렬 정보 참조하기

- `Mat::type()` : Mat 객체에 저장된 영상이 1채널 그레이스케일 영상인지 혹은 3채널 컬러 영상인지를 확인할때 사용
- `Mat::type()` 함수의 반환값이 `CV_8UC1`이면 그레이스케일 영상으로 간주하고, `CV_8UC3`이면 3채널 트루컬러 영상이라고 생각할 수 있음

```
if (img1.type() == CV_8UC1)
    cout << "img1 is a grayscale image." << endl;
else if (img1.type() == CV_8UC3)
    cout << "img1 is a truecolor image." << endl;

img1 is a truecolor image.
```

행렬 정보 참조하기

```
static inline  
std::ostream& operator << (std::ostream& out, const Mat& mtx)
```

- **out** C++ 표준 출력 스트림 객체
- **mtx** 출력할 행렬
- **반환값** C++ 표준 출력 스트림 객체의 참조

- OpenCV는 << 연산자 재정의를 이용하여 행렬 원소를 출력하는 기능을 제공함
- << 연산자 왼쪽에는 std::cout을 적고, << 연산자 오른쪽에는 Mat 객체 변수 이름을 적으면 해당 행렬 원소가 모두 콘솔 창에 출력됨

행렬 정보 참조하기

- 예를 들어 작은 크기의 행렬 원소를 모두 화면에 출력하려면 다음과 같이 코드를 작성할 수 있음

```
float data[] = { 2.f, 1.414f, 3.f, 1.732f };
Mat mat1(2, 2, CV_32FC1, data);

std::cout << mat1 << std::endl;
```

- 이 행렬을 << 연산자를 이용하여 std::cout 표준 출력 스트림으로 전달하면 콘솔 창에 다음과 같은 형태로 행렬 원소 값이 나타남

```
[2, 1.414;
 3, 1.732]
```

코드3-11 : 행렬정보 출력

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat img1 = imread("lenna.bmp");
    cout << "Width: " << img1.cols << endl;
    cout << "Height: " << img1.rows << endl;
    cout << "Channels: " << img1.channels() << endl;
    if (img1.type() == CV_8UC1)
        cout << "img1 is a grayscale image." << endl;
    else if (img1.type() == CV_8UC3)
        cout << "img1 is a truecolor image." << endl;
    float data[] = { 2.f, 1.414f, 3.f, 1.732f };
    Mat mat1(2, 2, CV_32FC1, data);
    cout << "mat1:\n" << mat1 << endl;
    return 0;
}
```

Microsoft Visual Studio 디버그 콘솔

```
Width: 512
Height: 512
Channels: 3
img1 is a truecolor image.
mat1:
[2, 1.414;
 3, 1.732]
```

행렬 연산

- OpenCV는 Mat 클래스가 표현하는 행렬을 마치 수학 수식을 쓰듯이 사용할 수 있도록 다양한 연산자 재정의 함수를 제공함
- 이를 통하여 행렬과 행렬의 덧셈 또는 곱셈 연산을 수행할 수 있고, 행렬에 상수 값을 곱하는 연산을 수행할 수도 있음

행렬 연산

- 행렬 사칙 연산을 위한 주요 연산자 재정의 함수는 다음과 같음

```
MatExpr operator + (const Mat& a, const Mat& b);
MatExpr operator + (const Mat& a, const Scalar& s);
MatExpr operator + (const Scalar& s, const Mat& a);
```

```
MatExpr operator - (const Mat& a, const Mat& b);
MatExpr operator - (const Mat& a, const Scalar& s);
MatExpr operator - (const Scalar& s, const Mat& a);
```

```
MatExpr operator - (const Mat& m);
```

```
MatExpr operator * (const Mat& a, const Mat& b);
MatExpr operator * (const Mat& a, double s);
MatExpr operator * (double s, const Mat& a);
```

```
MatExpr operator / (const Mat& a, const Mat& b);
MatExpr operator / (const Mat& a, double s);
MatExpr operator / (double s, const Mat& a);
```

행렬 연산

- 앞에 나열된 연산자 재정의 함수 인자로 사용된 s에는 Scalar 객체뿐만 아니라 **uchar, int, float, double** 같은 **C/C++ 기본 자료형을 전달할 수 있음**
- 연산자 재정의 함수의 반환형으로 사용된 MatExpr 타입은 OpenCV에서 행렬 연산을 표현하는 용도로 사용되는 행렬 표현식 클래스임
- MatExpr 클래스는 Mat 객체와 Scalar 객체, 그리고 int, float, double 같은 C/C++ 기본 자료형으로 구성된 행렬 연산을 표현함
- 복사 생성자 또는 대입 연산자에 의해 Mat 객체로 변환 될 수 있음
- 앞에 나열된 연산자 재정의 함수의 실행 결과는 모두 Mat 타입의 변수에 저장할 수 있음

행렬 연산 : 덧셈, 뺄셈

- 표 3-3에서 mat1, mat2, mat3은 Mat 클래스 타입의 변수이고, s1은 Scalar 타입의 변수
- s1 대신에 uchar, int, float, double 같은 기본 자료형을 사용가능

표 3-3 Mat 클래스 행렬 연산 예제 코드와 의미

행렬의 사칙 연산 예제 코드	설명
<pre>mat3 = mat1 + mat2; mat3 = mat1 - mat2;</pre>	mat1과 mat2 행렬 사이의 덧셈 및 뺄셈 연산을 수행합니다. ¹²
<pre>mat3 = mat1 + s1; mat3 = mat1 - s1; mat3 = s1 + mat1; mat3 = s1 - mat1;</pre>	mat1 행렬의 각 원소와 스칼라 값 s1 사이의 덧셈 및 뺄셈 연산을 수행합니다.

행렬 연산 : 곱셈, 나눗셈

- 표 3-3에서 mat1, mat2, mat3은 Mat 클래스 타입의 변수이고, d1은 double 자료형의 변수를 나타냄
- d1 대신에 uchar, int, float 같은 자료형을 사용할 수 있음, 다만 d1 위치에는 Scalar 객체를 사용할 수 없음

표 3-3 Mat 클래스 행렬 연산 예제 코드와 의미

행렬의 사칙 연산 예제 코드	설명
<code>mat3 = -mat1;</code>	mat1 행렬의 각 원소에 -1을 곱합니다.
<code>mat3 = mat1 * mat2;</code>	mat1과 mat2 행렬의 곱셈 연산을 수행합니다.
<code>mat3 = mat1 * d1;</code> <code>mat3 = d1 * mat1;</code>	mat1 행렬의 각 원소에 실수 d1을 곱합니다.
<code>mat3 = mat1 / mat2;</code>	mat1과 mat2 행렬의 같은 위치 원소끼리 나눗셈 연산을 수행합니다.
<code>mat3 = mat1 / d1;</code> <code>mat3 = d1 / mat1;</code>	mat1 행렬의 각 원소와 실수 d1끼리 나눗셈 연산을 수행합니다.

행렬 연산

```
MatExpr Mat::mul(InputArray m, double scale=1) const;
```

- `m` *this 행렬과 크기 및 타입이 같은 행렬, 또는 행렬 표현식
- `scale` 추가적으로 곱할 값
- 반환값 두 행렬의 같은 위치 원소끼리 곱셈한 결과 행렬 표현식

- 표 3-3에 나열된 행렬 연산 예제 중에서 * 연산자를 사용하는 곱셈 연산은 행렬의 수학적 곱셈 연산을 의미함
- 만약 두 행렬에서 같은 위치에 있는 원소끼리 곱셈 연산을 수행하려면 `Mat::mul()` 멤버 함수를 사용해야 함

행렬 연산

```
MatExpr Mat::inv(int method=DECOMP_LU) const;
```

- **method** 역행렬 계산 방법. DECOMP_LU, DECOMP_SVD, DECOMP_EIG, DECOMP_CHOLESKY 중 하나를 지정할 수 있습니다.
- **반환값** 역행렬에 대한 행렬 표현식

- OpenCV에서 행렬의 역행렬을 구할 때에는 Mat::inv() 멤버 함수를 사용함
- Mat::inv() 함수는 method 인자를 통해 역행렬 계산 방법을 지정할 수 있음
- 역행렬이 존재하는 일반적인 행렬이라면 가우스 소거법을 사용하는 DECOMP_LU를 사용할 수 있으며, 이 값은 기본값으로 지정되어 있으므로 생략할 수 있음

행렬 연산

```
MatExpr Mat::t() const;
```

• 반환값 전치 행렬에 대한 행렬 표현식

- 행렬의 행과 열을 서로 교환해서 만드는 전치 행렬(transpose matrix)을 구하는 함수
- 만약 전치 행렬 연산의 입력 행렬이 정방 행렬(square matrix)이면 단순히 행렬 원소의 행과 열 위치만 서로 바꿈
- 입력 행렬 크기가 $M \times N$ 이고 $M \neq N$ 이라면 `Mat::t()` 함수에 의해 구해지는 전치 행렬의 크기는 $N \times M$ 으로 변경됨

예제 1: 행렬 연산

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    float data[] = { 1, 1, 2, 3 };
    Mat mat1(2, 2, CV_32FC1, data);
    cout << "mat1:\n" << mat1 << endl;

    Mat mat2;
    mat2 = mat1+Scalar(1.5);           // Mat객체 + Scalar객체
    cout << "덧셈:\n" << mat2 << endl;
    mat2 = mat1+1.5;                   // Mat객체 + 실수
    cout << "덧셈:\n" << mat2 << endl;
    mat2 = mat1+10;                    // Mat객체 + 정수
    cout << "덧셈:\n" << mat2 << endl;
```

예제 1: 행렬 연산

```
//mat2 = mat1*Scalar(1.5);           // Mat객체*Scalar객체 불가능
//cout << "곱셈:\n " << mat2 << endl;
mat2 = mat1*1.5;                       // Mat객체*실수 가능
cout << "곱셈:\n " << mat2 << endl;
mat2 = mat1*10;                        // Mat객체*정수 가능
cout << "곱셈:\n" << mat2 << endl;

return 0;

}
```

예제 1: 행렬 연산

```
Microsoft Visual Studio 디버그 콘솔

mat1:
[1, 1;
 2, 3]
덧셈:
[2.5, 2.5;
 3.5, 4.5]
덧셈:
[2.5, 2.5;
 3.5, 4.5]
덧셈:
[11, 11;
 12, 13]
곱셈:
[1.5, 1.5;
 3, 4.5]
곱셈:
[10, 10;
 20, 30]
```

예제2: 그레이스케일영상 밝기 조절

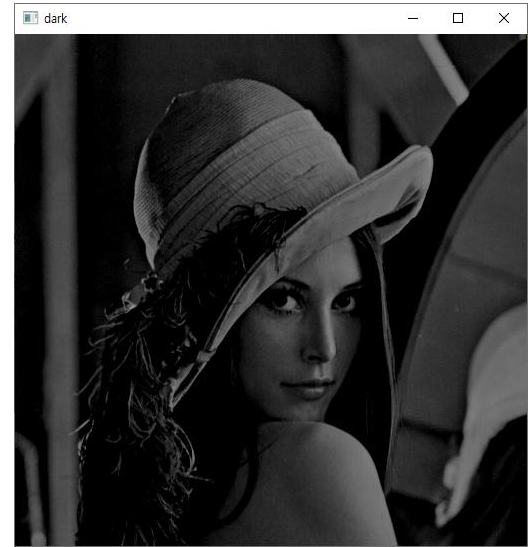
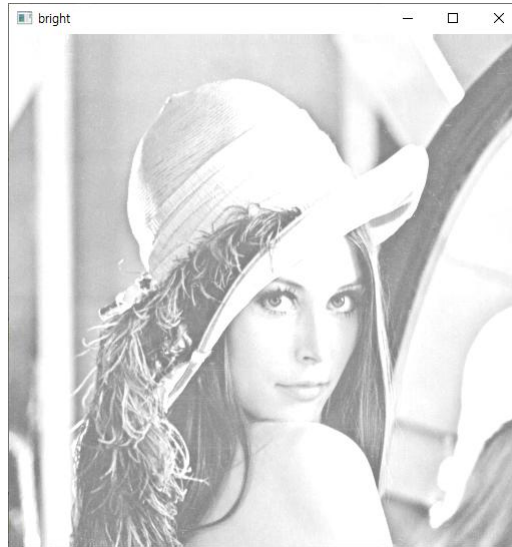
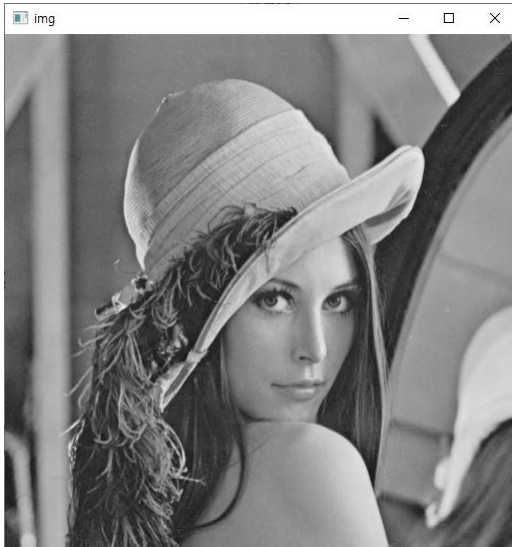
```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
int main()
{
    Mat img = imread("lenna.bmp", IMREAD_GRAYSCALE);
    imshow("img", img);

    Mat bright = img + 100;
    //Mat bright = img + Scalar(100);
    imshow("bright", bright);

    Mat dark = img - 100;
    //Mat dark = img - Scalar(100);
    imshow("dark", dark);

    waitKey(); return 0;
}
```

예제2: 그레이스케일영상 밝기 조절



예제3: 컬러영상 밝기 조절

```
#include <opencv2/opencv.hpp>
using namespace cv;
int main()
{
    Mat img = imread("lenna.bmp");
    imshow("img", img);

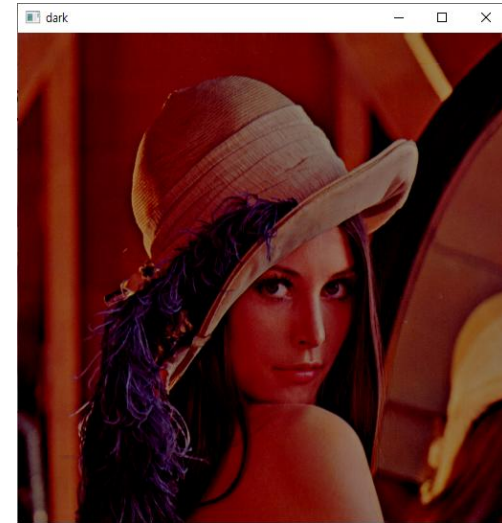
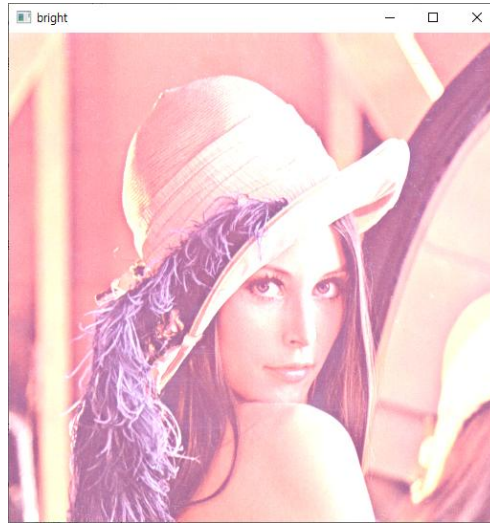
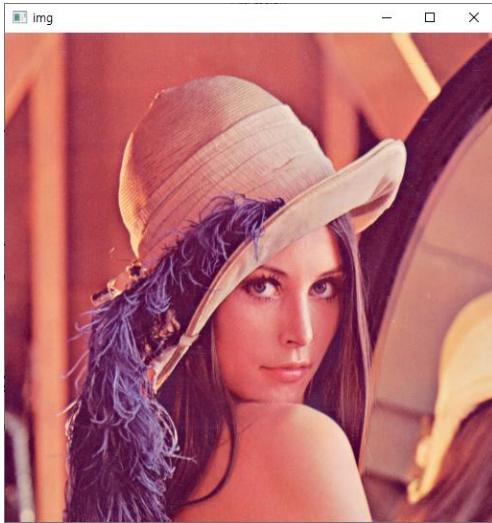
    Mat bright = img + Scalar(100,100,100);
    //Mat bright = img + 100;      //Blue값만 커짐
    imshow("bright", bright);

    Mat dark = img - Scalar(100,100,100);
    //Mat dark = img - 100;      //Blue값만 작아짐

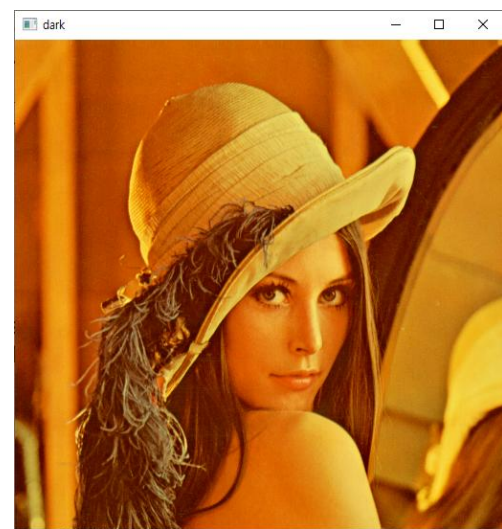
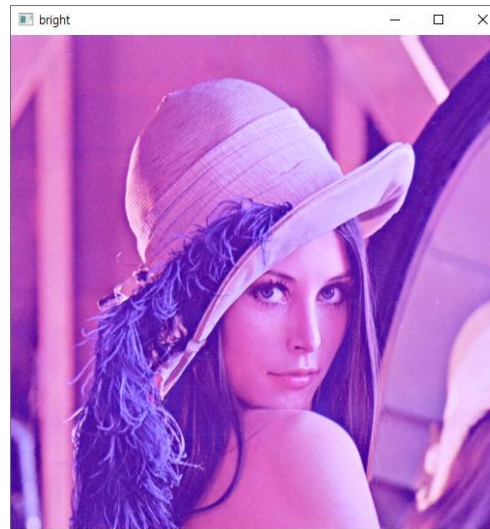
    imshow("dark", dark);

    waitKey(); return 0;
}
```


예제3: 컬러영상 밝기 조절



정수형 상수를
연산한 경우



코드3-12 : 행렬연산

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    float data[] = { 1, 1, 2, 3 };
    Mat mat1(2, 2, CV_32FC1, data);
    cout << "mat1:\n" << mat1 << endl;

    Mat mat2 = mat1.inv();
    cout << "mat2:\n" << mat2 << endl;
    cout << "mat1.t():\n" << mat1.t() << endl;
    cout << "mat1 + 3:\n" << mat1 + 3 << endl;
    cout << "mat1 + mat2:\n" << mat1 + mat2 << endl;
    cout << "mat1 * mat2:\n" << mat1 * mat2 << endl;

    return 0;
}
```

코드3-12 : 행렬연산

```
Microsoft Visual Studio 디버그 ...  
mat1:  
[1, 1;  
 2, 3]  
mat2:  
[3, -1;  
 -2, 1]  
mat1.t():  
[1, 2;  
 1, 3]  
mat1 + 3:  
[4, 4;  
 5, 6]  
mat1 + mat2:  
[4, 0;  
 0, 4]  
mat1 * mat2:  
[1, 0;  
 0, 1]
```

크기 및 타입 변환 함수

```
void Mat::convertTo( OutputArray m, int rtype, double alpha=1, double beta=0 ) const;
```

```
Mat Mat::reshape(int cn, int rows=0) const;
```

```
void Mat::resize(size_t sz);
```

```
void Mat::resize(size_t sz, const Scalar& s);
```

- 자세한 설명은 생략함, 교재 참고할 것

실습과제 1

- 휴대폰 카메라로 사진을 촬영하여 저장한 후 imread 함수를 이용하여 Mat 객체에 저장하고 영상의 행의 수, 열의 수, 채널수, type을 화면에 출력하는 코드를 작성하시오.
- type은 기호상수로 표시하라.(예: CV_8UC3 등)

실습과제2

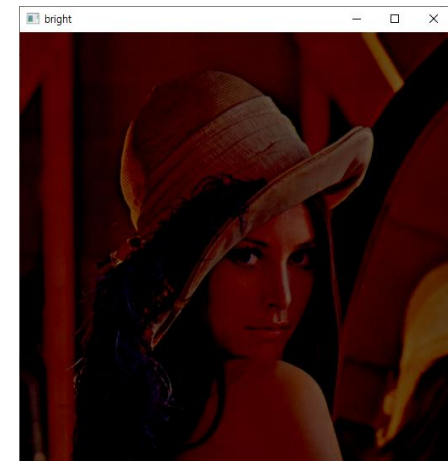
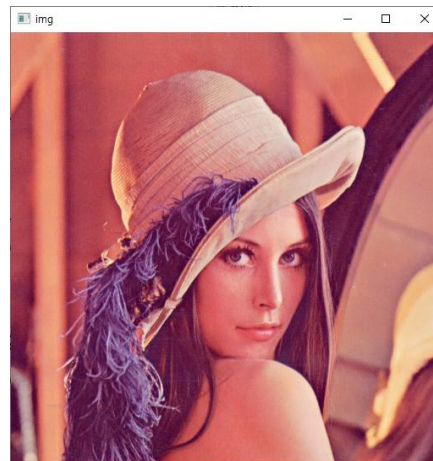
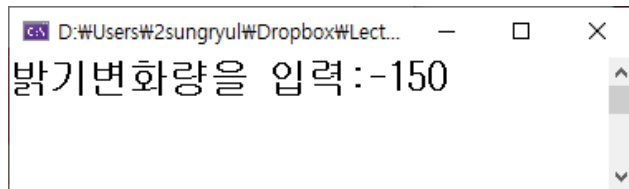
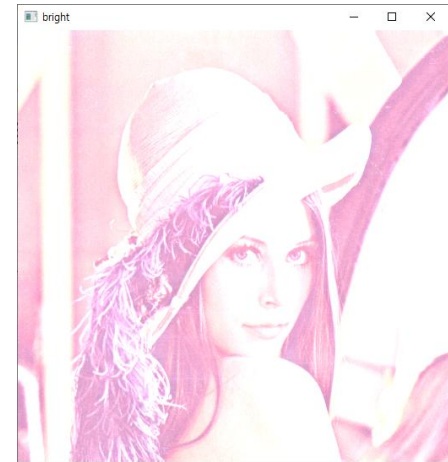
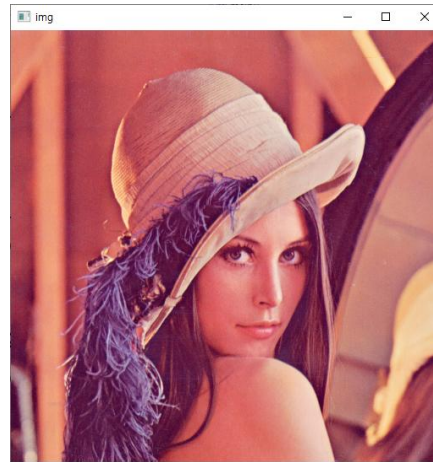
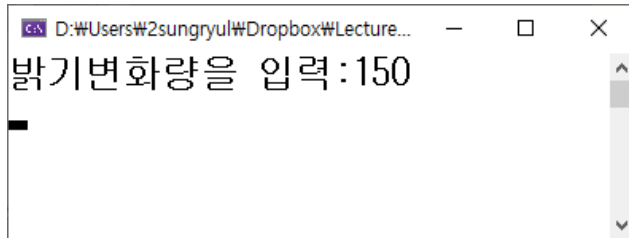
- 행렬연산을 이용하여 아래 수식에서 행렬 X 를 구하는 프로그램을 작성하시오. 행렬 X 를 화면에 출력하라.
- 역행렬 B^{-1} 은 `Mat::inv` 함수를 사용할 것

$$A = \begin{pmatrix} 1 & 3 \\ -4 & 2 \end{pmatrix}, B = \begin{pmatrix} 2 & 3 \\ 0 & 5 \end{pmatrix}, C = \begin{pmatrix} -2 & -2 \\ -2 & -3 \end{pmatrix}$$

$$X = 3 * A + B^{-1} + 10 * C - 5$$

실습과제3

- 행렬 연산을 이용하여 영상의 밝기를 수정하는 프로그램을 작성하시오.



과제 제출 방법

- 소스파일, 라인단위의 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.